

融合特征匹配与异常行为分析的 网络攻击检测系统

——总体设计报告——

项目名称 融合特征匹配与异常行为分析的网络攻击检测系统

组 长 韩宇飞 524031910172

成 员 李哲 524031910017

曾子恒 523010910022

陈志恒 524031910111

姜新晨 524031910769

学 院 计算机学院

指导教师 陈秀真 姚立红

日 期 2026 年 7 月 17 日

目录

- 1 系统需求分析 1
 - 1.1 安全需求 1
 - 1.1.1 已知 Web 攻击识别 1
 - 1.1.2 暴力破解行为发现 1
 - 1.1.3 网络扫描与横向扩散识别 1
 - 1.1.4 异常外联与 C2 通信检测 1
 - 1.2 具体功能要求 1
 - 1.3 开发平台要求 2
 - 1.4 运行环境要求 2
- 2 计划进度安排 3
- 3 系统总体架构 4
 - 3.1 平台架构 4
 - 3.2 功能架构与数据流 6
- 4 系统功能模块一：数据捕获与协议解析（李哲） 6
 - 4.1 模块结构与调用关系 6
 - 4.2 报文捕获子模块（packet_capture.py） 7
 - 4.3 协议解析与流重组子模块 8
 - 4.4 Mock 数据生成器（mock_generator.py） 9
- 5 系统功能模块二：特征行为检测引擎（曾子恒） 10
 - 5.1 模块结构 10
 - 5.2 特征库管理（signature_db.py） 10
 - 5.3 匹配引擎（matcher.py） 10
- 6 系统功能模块三：暴力破解检测模块（陈志恒） 11
 - 6.1 模块结构 11
 - 6.2 连接尝试收集（_collect_attempts） 11
 - 6.3 滑动窗口密度统计（_max_count_in_window） 12
 - 6.4 告警产出 14
- 7 系统功能模块四：异常行为检测模块（姜新晨） 14
 - 7.1 模块结构 14
 - 7.2 基线配置（baseline.py） 14
 - 7.3 端口扫描检测 15
 - 7.4 异常外联检测 15

8	系统功能模块五：告警汇总与监控 GUI（韩宇飞）	16
8.1	模块结构	16
8.2	行为关联汇总（aggregator.py）	17
8.3	图形监控界面（gui.py）	19
9	接口设计	20
9.1	报文记录格式（Packet Record）	20
9.2	统一告警格式（Alert Record）	21
9.3	统一函数签名	22
9.4	统一日志规范	22
10	开发平台	23
10.1	软件平台	23
10.2	硬件平台	23
11	系统出错处理	24

1 系统需求分析

1.1 安全需求

本系统面向小型网络环境（如实验室局域网、校内子网段），需解决以下四类安全检测问题：

1.1.1 已知 Web 攻击识别

网络中存在大量具有固定特征的攻击行为，如 SQL 注入（UNION SELECT、' OR 1=1）、跨站脚本 XSS（<script> 标签注入）、恶意系统命令执行（/bin/cat /etc/passwd）等。这些攻击以 HTTP 请求为载体，传统包过滤防火墙工作在网络层/传输层，无法检测应用层载荷中的恶意特征。系统需内置一个可维护、可扩展的攻击特征库，对实时流量的应用层 payload 进行多模式匹配，且支持字面匹配（literal）和正则表达式匹配（regex）两种模式。

1.1.2 暴力破解行为发现

面向 SSH（22 端口）、FTP（21 端口）、数据库（3306、5432、6379 端口）、Web 登录（8080、8443 端口）等认证服务的暴力密码枚举攻击极为常见。攻击者在短时间内以极高频率发起连接尝试，表现为同一源 IP 在几十秒内对同一目标登录端口发起数十次 TCP SYN。系统需基于时间窗口的滑动密度统计自动发现此类行为，并结合服务端 RST 拒绝响应的次数佐证认证失败。

1.1.3 网络扫描与横向扩散识别

攻击者在渗透阶段通常在信息搜集后进行端口扫描（发现开放服务）和内网横向移动（跳板攻击）。端口扫描表现为单一源 IP 在短时间内访问大量不同目标端口（24 端口以上/60 秒），横向扩散表现为单台内网主机在数分钟内访问多个不同内网 IP。系统需建立正常流量行为基线，识别上述显著偏离的探测行为。

1.1.4 异常外联与 C2 通信检测

受控主机（僵尸节点）可能向外部 C2 服务器发起隐蔽通信。这类连接在正常业务流量中不常见——内网主机通常不主动访问陌生公网 IP。系统需维护内网 IP 段白名单，识别内网主机向不在白名单中的公网 IP 发起的出站连接。

1.2 具体功能要求

系统按功能分解为五个模块，对应五位成员分工开发：

表 1: 系统功能模块划分

编号	模块名称	负责人	核心功能
M1	数据包捕获与协议解析	李哲	实时/离线抓包、协议字段解析、TCP 流重组、payload 提取、TLS 基础识别
M2	特征行为检测引擎	曾子恒	特征库加载、字面/正则双模式匹配、同源同类 60s 窗口聚合为行为告警
M3	暴力破解检测模块	陈志恒	SYN 连接收集 + 退化 flow_id 计数、双指针滑动窗口密度统计、RST 拒绝佐证
M4	异常行为检测模块	姜新晨	端口扫描检测（同源端口多样性）、异常外联检测（内网 IP 白名单匹配）
M5	告警汇总与监控 GUI	韩宇飞	多源告警合并/排序/去重、攻击行为关联、三页签 GUI 面板

各模块通过统一的 JSON 接口（报文记录格式 + 告警记录格式）松耦合通信。Phase1-3 阶段所有检测模块的输入统一来自 `mock_data/mock_packets.json`（108 条覆盖 7 类场景的模拟报文），Phase4 起替换为真实抓包输出。

1.3 开发平台要求

表 2: 开发平台选型

项目	选型	选型理由
开发语言	Python $\geq$ 3.9	生态丰富, scapy/pytest/tkinter 均为主流库
抓包库	scapy $\geq$ 2.5.0	Python 原生, 支持实时抓包与 pcap 离线解析
GUI 框架	tkinter（标准库）	零依赖, ttk 主题化组件
测试框架	pytest $\geq$ 7.0	简洁断言, fixture 复用 mock 数据
版本控制	Git + GitHub Flow	Feature 分支开发, PR 合并
代码格式化	black + isort	推荐, 统一代码风格
AI 辅助	Claude Code / Cursor / Trae / 通义灵码	提升开发效率, 尤其在算法实现和测试生成方面

1.4 运行环境要求

软件环境:

- 操作系统：Linux（推荐 Ubuntu 22.04 LTS）或 Windows 10/11 + WSL2
- Python 3.9 及以上版本
- pip 依赖：scapy>=2.5.0, pytest>=7.0.0
- 实时抓包场景需安装 Npcap（Windows）或 libpcap（Linux）

硬件环境：

- 普通 PC 或笔记本，CPU 双核及以上，内存 ≥ 4GB
- 网卡支持混杂模式（Promiscuous Mode），用于实时抓包
- 若在虚拟机中运行，需将网络适配器配置为桥接模式

权限要求：

- 实时抓包需 root 权限（Linux）或管理员权限（Windows）
- 仅读取 pcap 文件则无需提权

2 计划进度安排

项目自 2026 年 7 月 8 日启动，至 8 月 5 日最终提交，共四周。

表 3: 项目进度表

日期	里程碑	状态
7/9(四)–7/12(日)	李哲交付 mock 数据 (108 条) + capture 模块	已完成
7/13(一)–7/18(六)	四人完成核心检测逻辑	进行中
7/20(一)	接口对齐检查 (aggregator 校验各模块告警格式)	待进行
7/21(二)–7/22(三)	单元测试编写 + 异常处理补充	待进行
7/23(四)–7/25(六)	全链路联调 (真实抓包替换 mock) + PR 合入 main	待进行
7/26(日)–7/28(二)	PPT 制作 + 演示数据准备 + 内部预演	待进行
7/29(三)	课堂项目分享汇报	待进行
7/30(四)–8/1(六)	结题报告撰写 + 源码整理	待进行
8/2(日)	提前提交 (比 8/5 截止早 3 天)	待进行
8/3(一)–8/4(二)	缓冲时间，应对突发问题	待进行
8/5(三)	官方结题截止	待进行

截至 7 月 17 日，李哲 (capture)、陈志恒 (bruteforce)、韩宇飞 (aggregator+GUI) 三个模块已完成并合入 main 分支。曾子恒 (signature) 和姜新晨 (anomaly) 两个检测模块正在开发中。

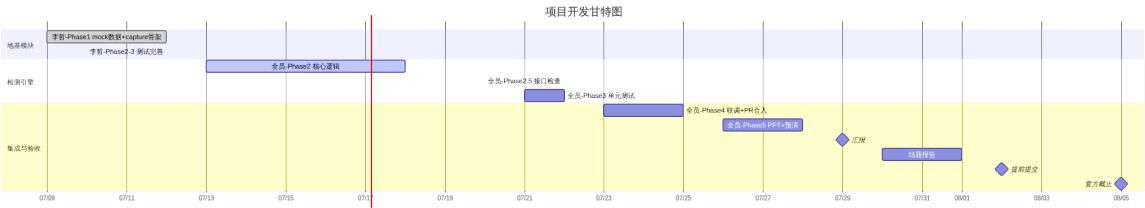


图 1: 项目开发甘特图

3 系统总体架构

3.1 平台架构

系统采用自底向上的四层分层架构：数据捕获层 → 检测引擎层 → 汇总关联层 → 展示层。各层之间通过统一的 JSON 数据格式解耦。

表 4: 四层架构各层职责

层级	名称	职责与产出
L1	数据捕获层	从网卡/pcap 获取报文，解析为标准化记录，完成 TCP 流重组。 产出： Packet Record []
L2	检测引擎层	三个引擎并行消费报文，从不同维度检测攻击行为。 产出： Alert Record []
L3	汇总关联层	合并多源告警，行为关联，赋予统一 behavior_id。 产出： merged_alerts.json
L4	展示层	以攻击行为为粒度展示，筛选/搜索/统计/特征库管理。 产出： GUI 窗口

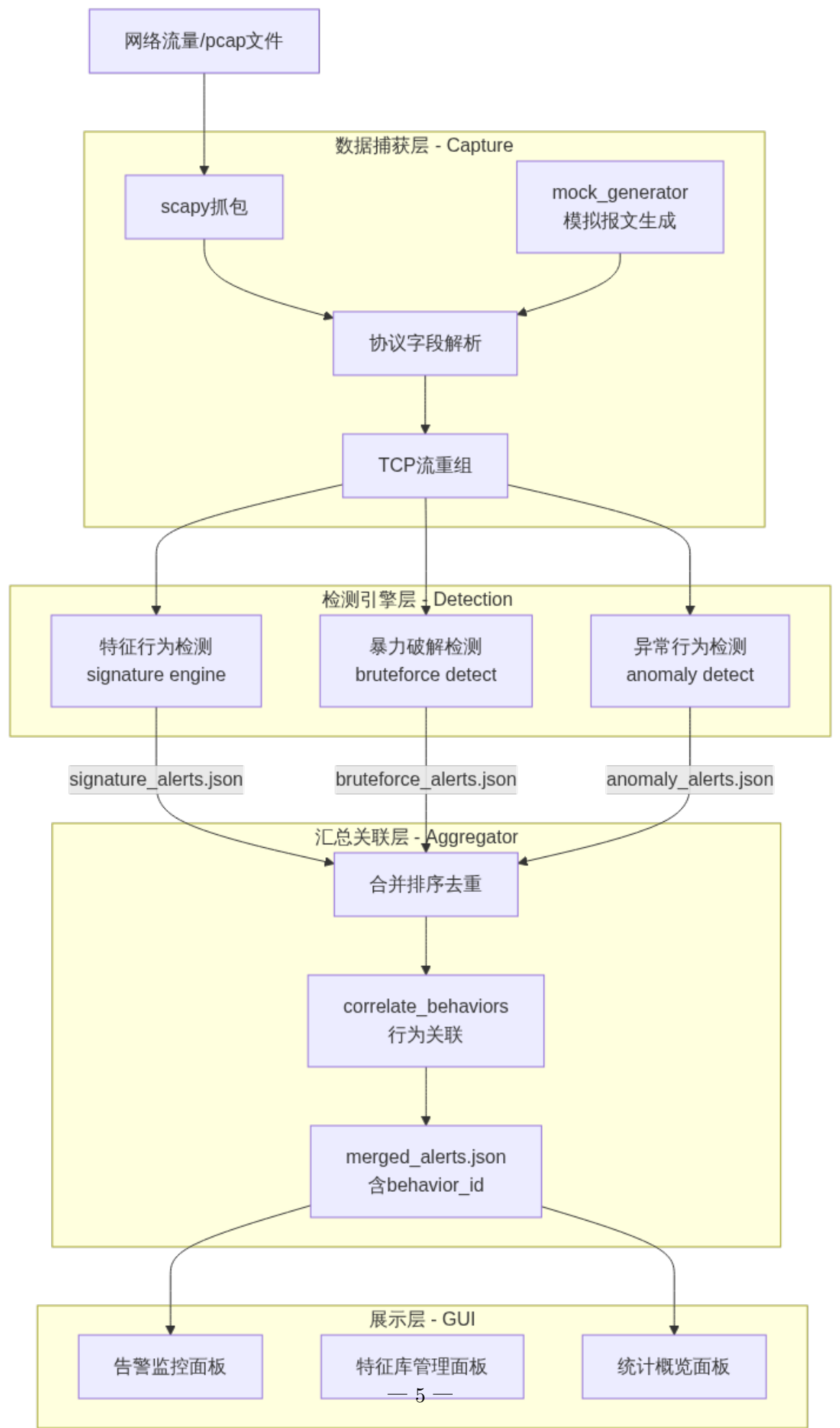


图 2: 系统四层平台架构

3.2 功能架构与数据流

系统核心数据流为：

数据包捕获 → TCP 重组/协议识别 → 并行检测（特征行为检测 || 暴力破解检测 || 异常行为检测）→ 行为告警关联与汇总 → 监控 GUI。

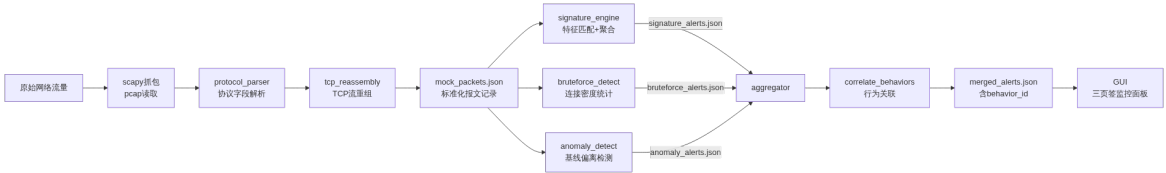


图 3: 系统数据流全景图

关键设计决策：

- 1. **并行检测架构：**三个检测引擎独立运行，彼此不依赖对方的输出。这允许任一引擎单独测试和部署，也允许未来增加新的检测引擎而不影响现有模块。
- 2. **统一的 JSON 接口：**所有模块通过两个标准 JSON Schema（Packet Record 和 Alert Record）通信。模块间不直接导入彼此的 Python 代码，仅通过 JSON 文件交换数据。这保证了即使独立开发者使用不同的代码风格或第三方库，也不会产生依赖冲突。
- 3. **行为聚合而非逐包告警：**无论是特征匹配、暴力破解还是异常检测，最终产出都以”攻击行为事件”为粒度。特征引擎对同源同类命中做窗口聚合，暴力破解按（src_ip, dst_ip, dst_port）分组，异常检测以偏离行为为单位。这使得 GUI 展示的告警数量可控、可读，避免告警洪流。
- 4. **CLI + 全链路两种运行模式：**每个检测模块提供独立的 python -m CLI 入口（--input / --output），便于单模块调试；同时 main.py 提供一键运行全链路的入口，用于集成测试和演示。

4 系统功能模块一：数据捕获与协议解析（李哲）

4.1 模块结构与调用关系

```
src/capture/
__init__.py
packet_capture.py      # 抓包入口：capture_live() / read_pcap()
protocol_parser.py     # 协议解析核心
tcp_reassembly.py      # TCP流重组
mock_generator.py      # Mock数据生成器
```

子模块调用关系：packet_capture → protocol_parser.parse_packet() → tcp_reassembly.reassemble() → 标准化记录列表。

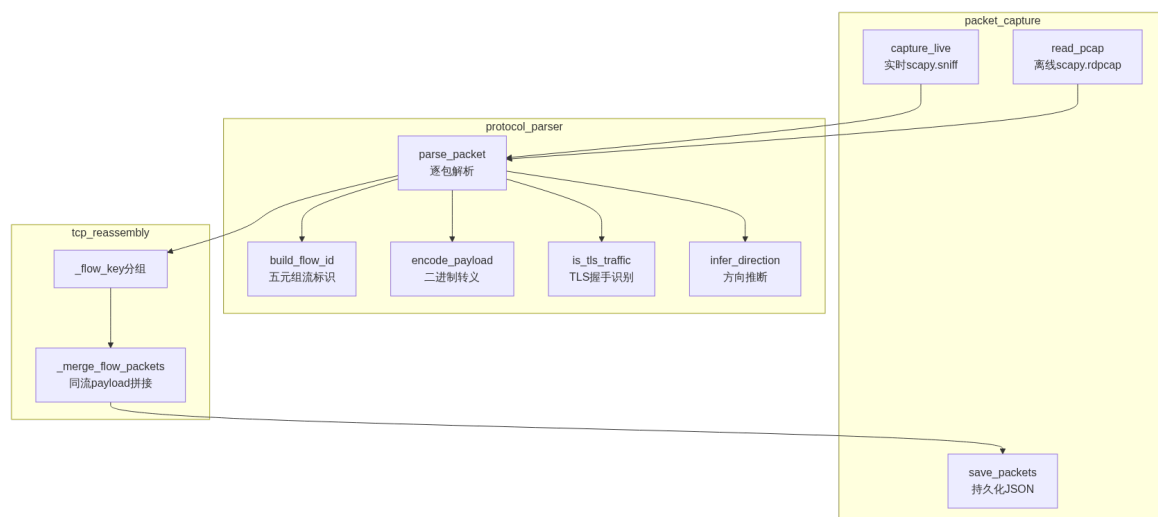


图 4: capture 模块内部调用关系

4.2 报文捕获子模块（packet_capture.py）

实时抓包 `capture_live(interface, count, timeout, do_reassemble):`

- 底层调用 `scapy.all.sniff(iface=interface, count=count, timeout=timeout)` 从指定网卡捕获原始报文
- `count=0` 时表示不限数量，由 `timeout` 参数控制抓包时长
- 捕获完成后逐包调用 `parse_packet()` 解析,可选调用 `reassemble()` 进行 TCP 重组
- `scapy` 未安装时打印 ERROR 日志并返回空列表，不崩溃

离线读取 `read_pcap(filepath, do_reassemble):`

- 调用 `scapy.all.rdpcap(filepath)` 读取 pcap/pcapng 文件
- 文件不存在时打印 WARNING 日志并返回空列表

CLI 入口:

```

# 离线模式
python -m src.capture.packet_capture \
    --pcap capture.pcap --output results/captured_packets.json
# 实时模式
python -m src.capture.packet_capture \
    --live --interface eth0 --count 100 --timeout 30
# 跳过TCP重组
python -m src.capture.packet_capture --pcap capture.pcap --no-reassemble

```

4.3 协议解析与流重组子模块

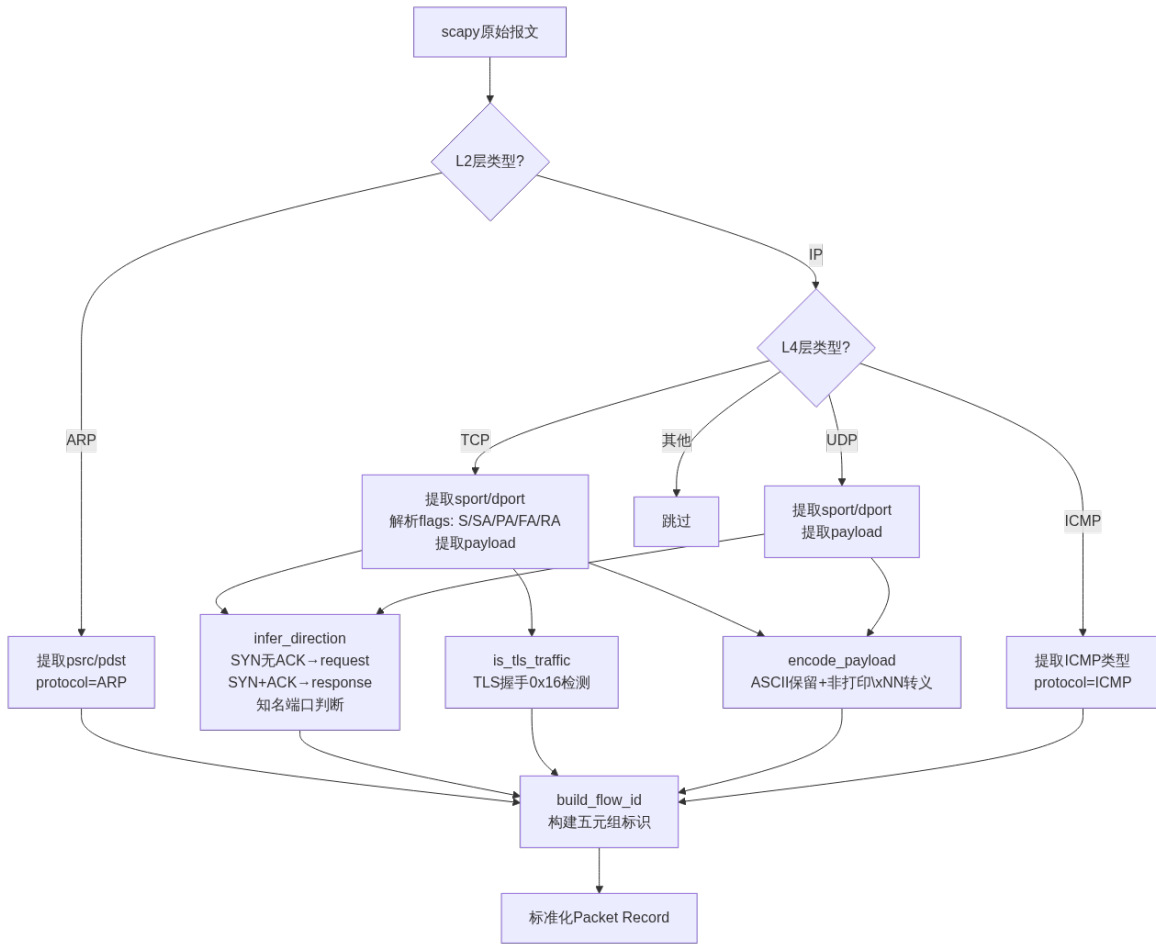


图 5: parse_packet 协议栈解析流程

关键字段解析规则:

- timestamp: 从 scapy 报文的 .time 属性提取, 格式化为 ISO8601 毫秒精度
- flow_id: {src_ip}:{src_port}->{dst_ip}:{dst_port}/{protocol}
- direction: 优先通过 TCP flags 判断 (SYN 无 ACK → request, SYN+ACK → response), 其次通过知名服务端口列表判断
- flags: 将 scapy 的 TCP flag 位映射为紧凑字符串 (F/S/R/P/A/U/E/C)
- payload: 可打印 ASCII(32-126) 和空格/TAB/回车/换行直接保留, 其余字节转为 \xNN 形式
- payload_len: 线路上原始 TCP/UDP 数据段的字节长度, 非转义后字符串长度

TLS 流量识别 (加分项) is_tls_traffic(payload, src_port, dst_port):

- 检测 TLS 记录层 ContentType=0x16(Handshake)且 Version ∈ {0x0300, 0x0301, 0x0302, 0x0303}
- 辅助检测: 当端口为 443/8443/465/993/995 时, 增强置信度判断
- 识别到 TLS 流量时打印 DEBUG 日志标记

TCP 流重组 `reassemble(packets):`

- 将报文按 `flow_id` 分组为 TCP 流（非 TCP 报文原样保留）
- 每条流内按 `timestamp` 排序后，区分数据报文（`payload_len>0`）和控制报文（`SYN/FIN/RST` 无数据）
- 数据报文合并：拼接所有 `payload` 字符串、累加 `payload_len`，以流中首条报文的时间戳和地址信息为代表
- 控制报文原样输出不合并（保留连接建拆的时序信息供检测模块使用）
- 输出按 `timestamp` 重新排序

4.4 Mock 数据生成器（`mock_generator.py`）

`generate_mock_packets()` 函数以 `datetime(2026,7,8,10,0,0)` 为基准时间，通过工厂函数 `_pkt()` 逐条构造报文，时间戳以 50ms 步进，确保每条报文时间唯一且有序。

表 5: Mock 数据场景覆盖对照表

场景	数量	关键特征	预期检出模块
正常 HTTP 流量	8 条	GET/POST 请求 + 响应	均不应告警
正常 SSH 流量	4 条	SSH banner+ 握手	均不应告警
正常 FTP 流量	6 条	USER/PASS/LIST 命令	均不应告警
正常 DNS+ICMP	4 条	UDP DNS + ICMP echo	均不应告警
SQL 注入特征	7 条	UNION SELECT, 'OR 1=1	signature
XSS 特征	6 条	<script>, <SCRIPT>	signature
木马/恶意命令	4 条	/bin/cat, /etc/passwd	signature
暴力破解 SSH	36 条	18 次 SYN+18 次 RST	bruteforce
端口扫描	24 条	24 个不同端口 SYN 探测	anomaly
异常外联	3 条	内网 IP 连陌生公网 IP	anomaly
正常 HTTPS	1 条	TLS ClientHello	均不应告警

合计：108 条，覆盖全部 7 类需求场景。

5 系统功能模块二：特征行为检测引擎（曾子恒）

5.1 模块结构

```
src/signature_engine/  
    __init__.py  
    signature_db.py    # 特征库管理  
    matcher.py         # 匹配引擎 + CLI
```

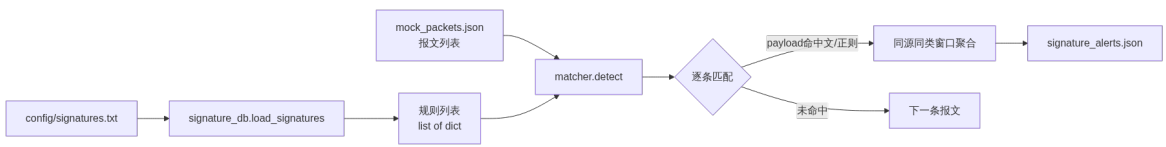


图 6: signature 模块结构与数据流

5.2 特征库管理（signature_db.py）

规则配置文件格式（config/signatures.txt）：

```
# 格式：规则ID | 攻击类型 | 匹配模式 | 特征串 | 适用协议 | 严重程度  
SIG-001 | SQL注入 | literal | UNION SELECT | HTTP | high  
SIG-002 | SQL注入 | literal | ' OR 1=1 | HTTP | high  
SIG-003 | XSS | regex | <script.*?>.*?</script> | HTTP | medium  
SIG-004 | 恶意命令 | literal | /bin/cat%20/etc/passwd | HTTP | high  
SIG-005 | 木马通信 | literal | /faxsurvey? | TCP | high
```

加载逻辑 load_signatures(filepath)：

- 按行读取，跳过空行和 # 开头的注释行
- 以 | 分隔符拆分为 6 个字段
- 字段数不等于 6 的行打印 WARNING 跳过，不中断加载
- 返回值：list[dict]，每条规则包含上述 6 个键

匹配模式说明：

- literal：大小写不敏感的 str.lower() in 子串匹配，性能 O(n.m)
- regex：使用 re.search(pattern, payload, re.IGNORECASE) 正则匹配
- 两者均可作为 baseline；高效多模式匹配（如 AC 自动机）是加分项

5.3 匹配引擎（matcher.py）

行为聚合规则：

- 分组键：(src_ip, category)

- 窗口：默认 60 秒
- 同一分组内时间间隔 $\leq$ 窗口的同类型告警共享一个 `behavior_id`
- 作用：将逐包命中聚合为一个攻击行为事件，而非逐条报 N 个告警

CLI 入口：

```
python -m src.signature_engine.matcher \  
    --input mock_data/mock_packets.json \  
    --output results/signature_alerts.json
```

6 系统功能模块三：暴力破解检测模块（陈志恒）

6.1 模块结构

```
src/bruteforce_detect/  
    __init__.py  
    login_monitor.py    # 完整检测逻辑（单文件内聚）
```

单文件内聚设计：所有辅助函数和检测逻辑集中在一个模块中，外部仅需调用 `detect(packets)`。

6.2 连接尝试收集（`__collect_attempts`）

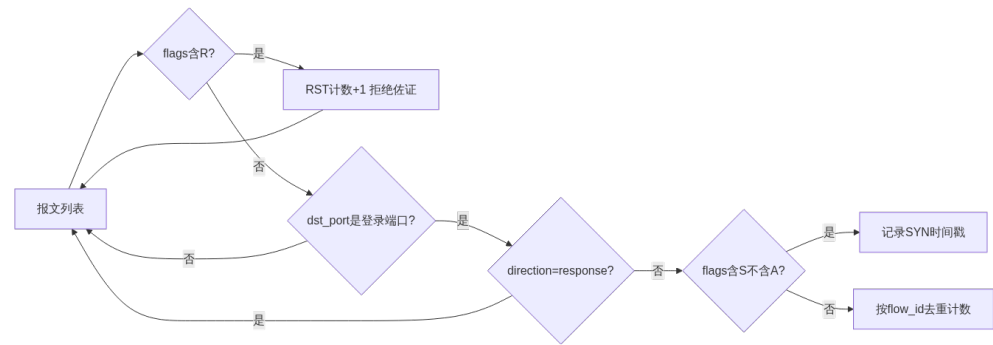


图 7: `__collect_attempts` 连接尝试收集流程

双模式计数策略：

1. 优先 SYN 计数：flags 含 “S” 且不含 “A” 的报文视为一次新建连接尝试。一次 SYN = 一次 TCP 握手发起，直接对应一次登录尝试。
2. 退化 flow_id 去重：当 flags 字段缺失时，退化为按不同 flow_id 计数，同一 flow_id 的多个数据包只计一次，防止重复计数。

RST 拒绝佐证：

- 监测方向为受害目标 $\rightarrow$ 攻击源的 RST 报文（flags 含 “R”，src_port 为登录端口）
- 统计值在告警中作为“连接被拒绝次数”出现，证明认证失败

6.3 滑动窗口密度统计 (__max_count_in_window)

双指针滑动窗口算法求任意 60 秒窗口内的最大连接尝试密度:

- 时间复杂度 $O(n)$, 空间复杂度 $O(1)$
- 维护 left/right 两个指针, right 向右扩展时 left 收缩确保窗口 $\leq \text{window_sec}$
- 返回 (窗口内最大事件数, 该窗口最后事件的时间戳)

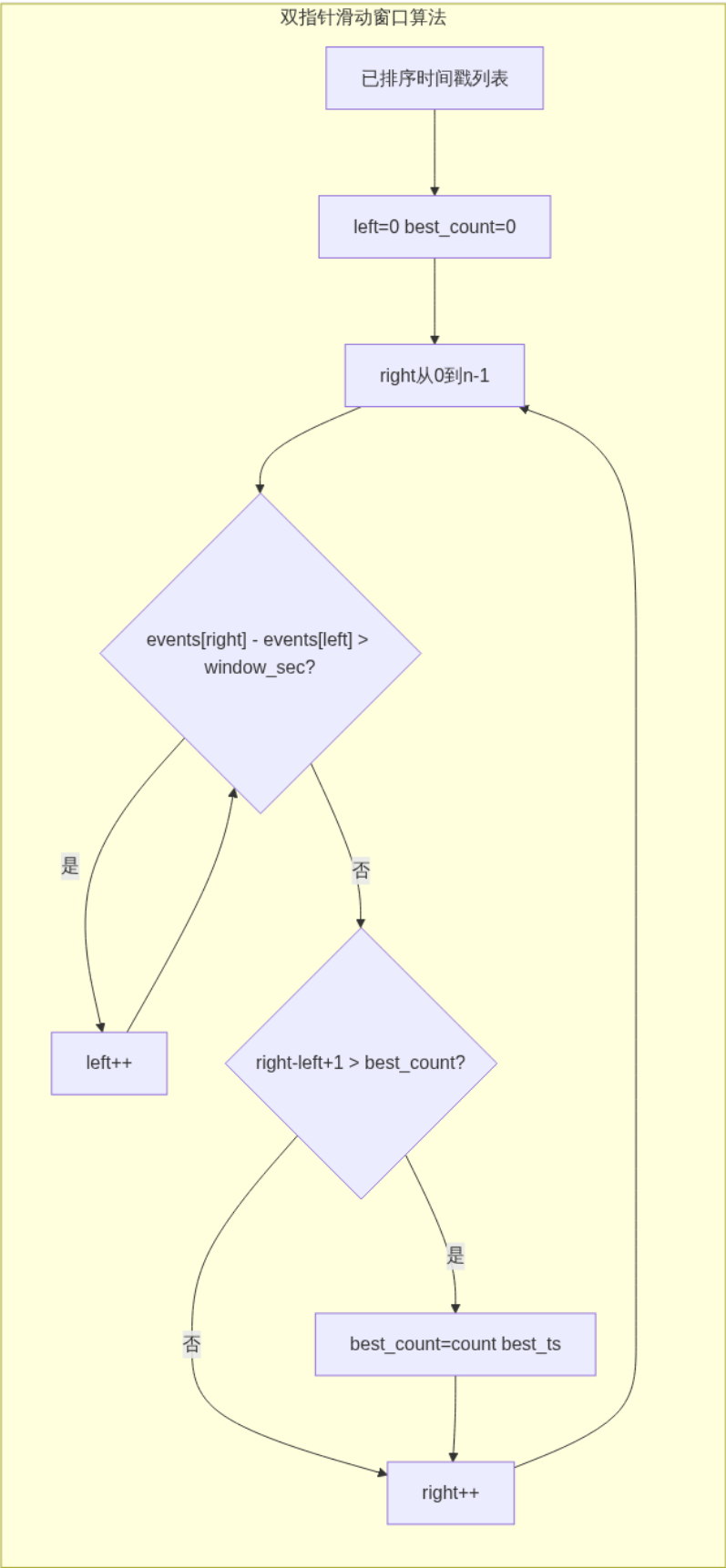


图 8: 双指针滑动窗口算法

6.4 告警产出

- **严重度分档**:窗口内尝试次数达到 threshold(默认 10)→ medium;达到 $2 \times \text{threshold}$ (20) → high
- **端口 → 服务名映射**:21→FTP, 22→SSH, 23→Telnet, 3389→RDP, 3306→MySQL, 5432→PostgreSQL, 6379→Redis
- **告警描述 (行为导向)**: “检测到针对 SSH 服务 (22 端口) 的暴力破解行为, 攻击源 X 在 60 秒内向 Y 发起 18 次连接尝试, 超出基线阈值 10 次, 其中 18 次连接被目标拒绝 (RST), 疑似认证失败”
- **behavior_id = alert_id** (每个分组即一个攻击行为事件)
- 输出字段集严格对齐 docs/interface_spec.md 的 Alert Record schema

CLI 入口:

```
python -m src.bruteforce_detect.login_monitor \
    --input mock_data/mock_packets.json \
    --output results/bruteforce_alerts.json \
    --window 60 --threshold 10
```

7 系统功能模块四：异常行为检测模块（姜新晨）

7.1 模块结构

src/anomaly_detect/

__init__.py

baseline.py # 基线配置加载

anomaly_detector.py # 端口扫描 + 异常外联 + CLI

7.2 基线配置 (baseline.py)

load_config(config_path) 从 config/baseline_config.json 读取阈值配置, 文件不存在时返回空字典并使用代码中默认值, 不崩溃。

配置文件结构:

```
{
  "port_scan": {
    "time_window_sec": 60,
    "unique_dst_port_threshold": 20
  },
  "external_connection": {
    "internal_networks": ["192.168.0.0/16", "10.0.0.0/8",
      "172.16.0.0/12"]
  },
  "lateral_movement": {
    "time_window_sec": 300,
```

```
"internal_dst_count_threshold": 10
}
}
```

7.3 端口扫描检测

以 `src_ip` 为分组维度,统计在 `time_window_sec` (默认 60 秒)窗口内该 IP 访问过的不同 `dst_port` 集合。当 `len(unique_dst_ports) >= unique_dst_port_threshold` (默认 20) 时产生告警。

- `dst_network` 字段填入被扫描的内网网段 CIDR 表示法
- `category = " 端口扫描"`, `detector = "anomaly"`
- `severity` 按偏离程度分级

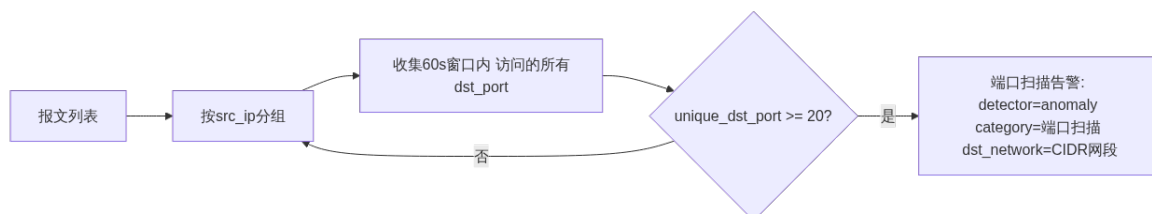


图 9: 端口扫描检测流程

7.4 异常外联检测

判断条件:

1. `src_ip` 属于内网 IP 段 (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16)
2. `dst_ip` 不属于上述任一内网段

满足条件的每个唯一 (`src_ip`, `dst_ip`) 对产生一条告警, `severity = "medium"`。

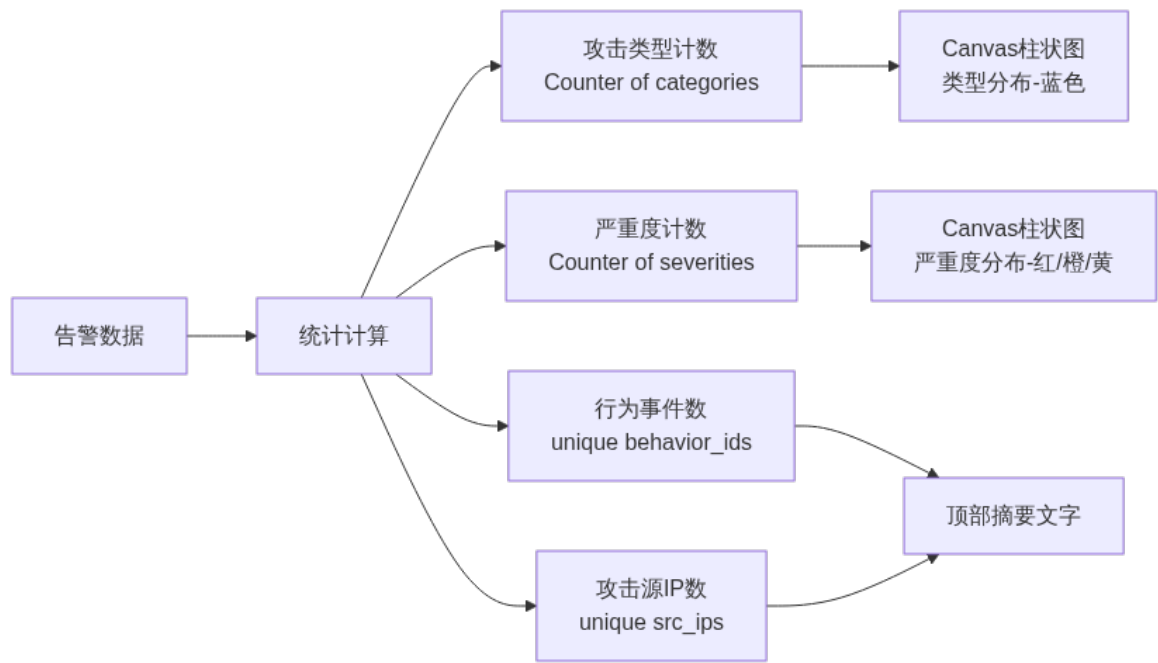


图 10: 异常外联检测流程

CLI 入口:

```
python -m src.anomaly_detect.anomaly_detector \
    --input mock_data/mock_packets.json \
    --output results/anomaly_alerts.json
```

8 系统功能模块五：告警汇总与监控 GUI（韩宇飞）

8.1 模块结构

```
src/gui_alert/
__init__.py
aggregator.py      # 汇总 + 去重 + 行为关联
gui.py            # tkinter 三页签 GUI
```

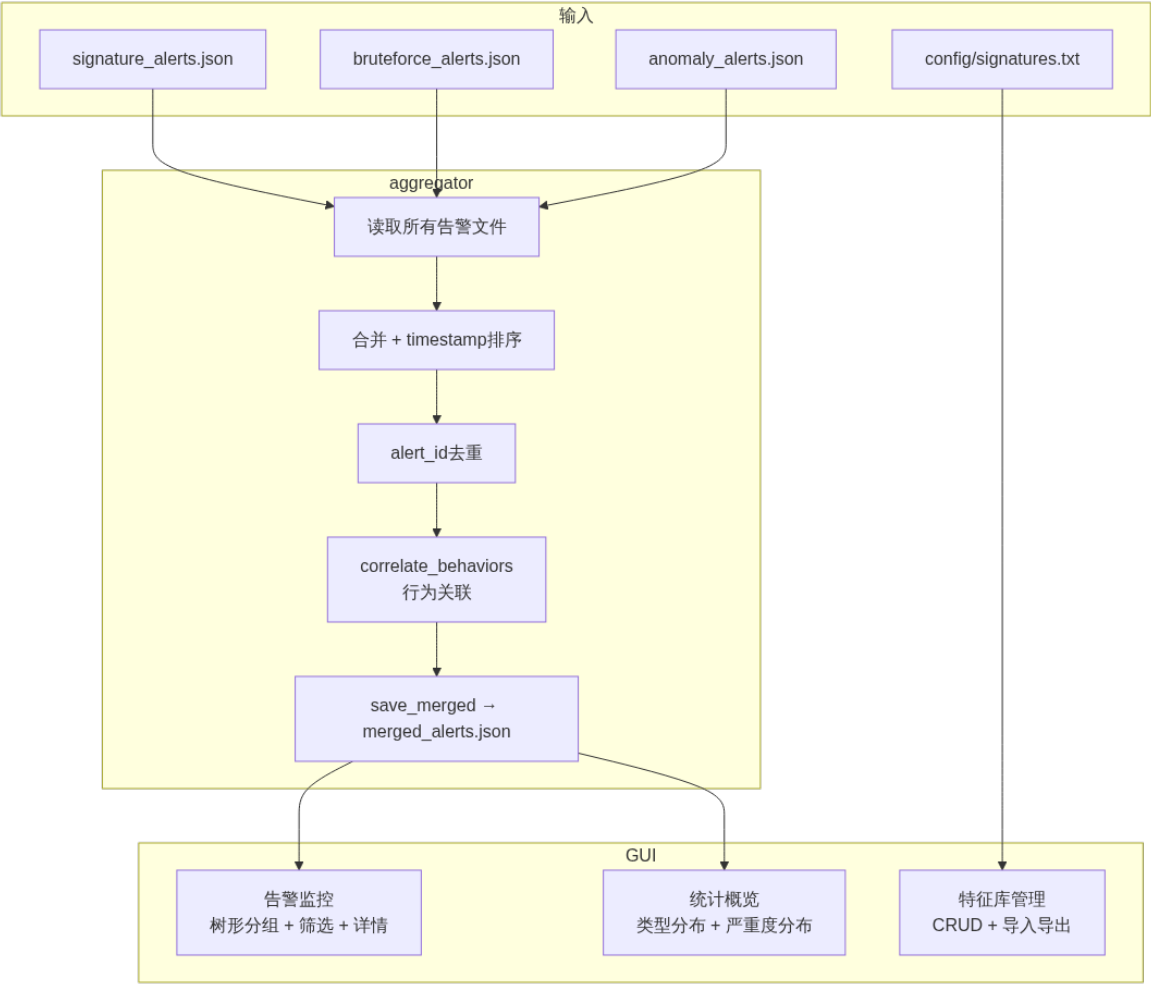


图 11: aggregator + GUI 模块结构与数据流

8.2 行为关联汇总 (aggregator.py)

aggregate(alert_files) 处理流程:

- 1. 加载: 遍历 alert_files 列表, 逐个 JSON 反序列化, 文件缺失/格式异常时打印 WARNING 跳过, 不中断。
- 2. 合并排序: 将所有告警按 timestamp 升序排列。
- 3. 去重: 以 alert_id 为键去重, 重复的 alert_id 仅保留首次出现者。
- 4. 行为关联: 调用 correlate_behaviors(alerts, time_window_sec=60)。

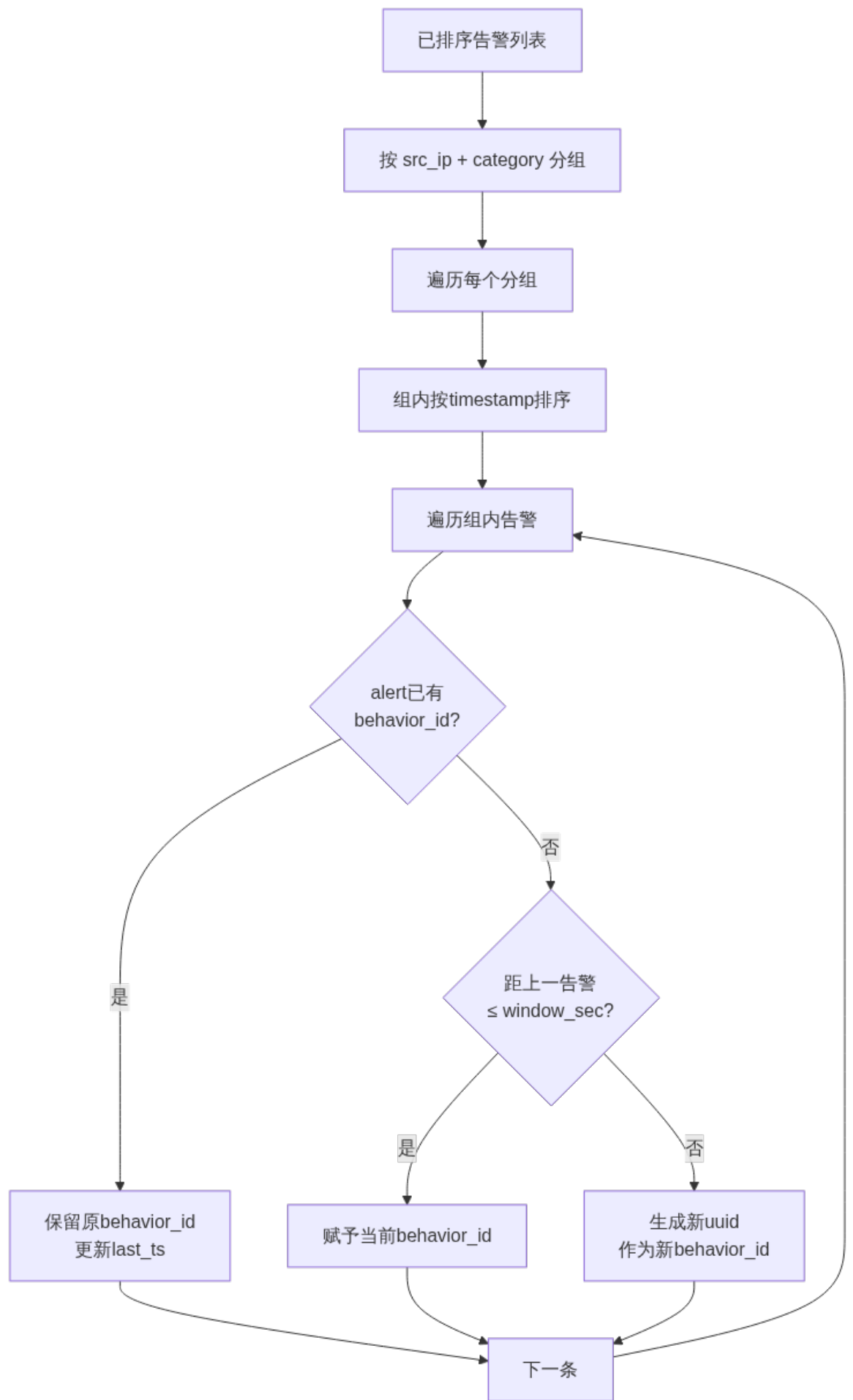


图 12: correlate_behaviors 行为关联算法

关联规则:

- 同一 `src_ip` + 同一 `category` + 时间间隔 ≤ 60 秒 $\rightarrow$ 同一 `behavior_id`
- 检测模块已自行标注 `behavior_id` 的告警, `aggregator` 保留原值不覆盖
- 跨 `detector` 的同源同类告警也会被关联

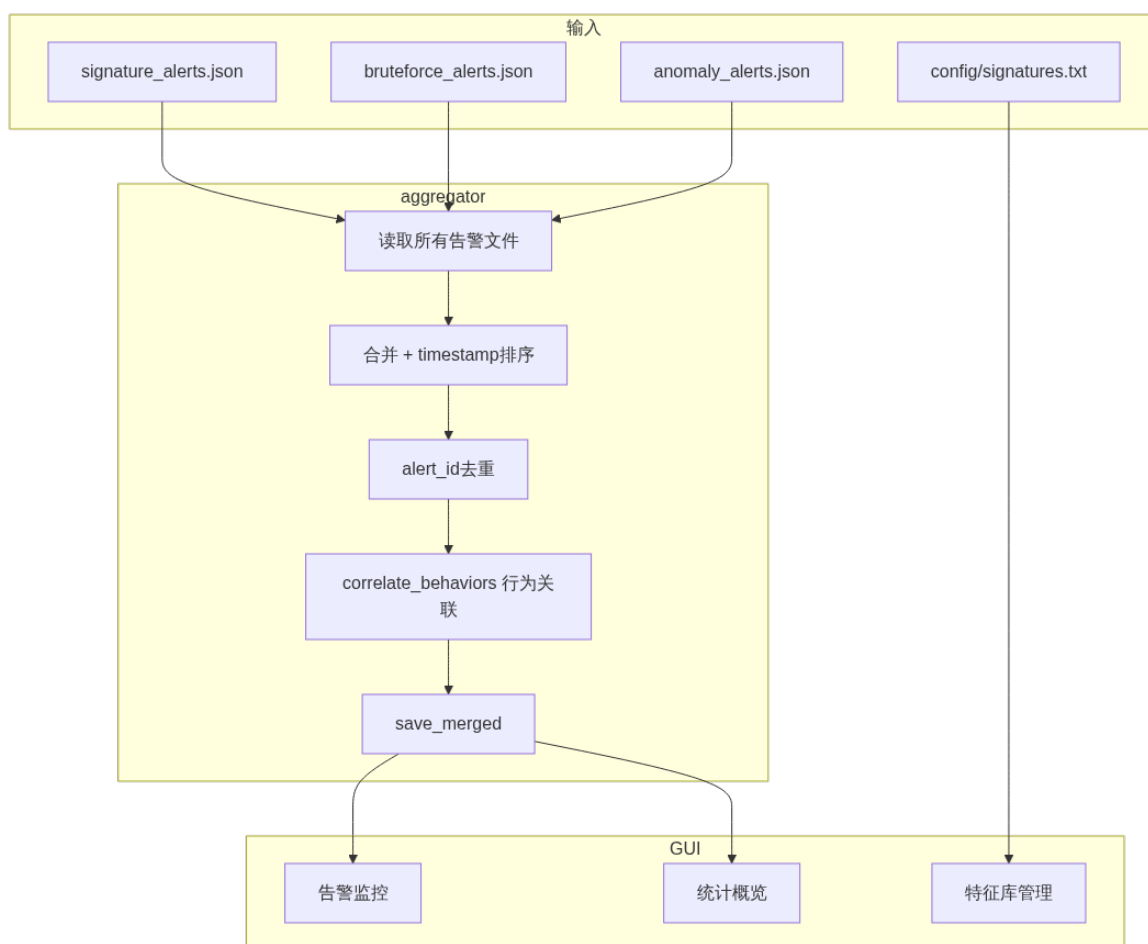
8.3 图形监控界面 (gui.py)

图 13: GUI 三页签结构

基于 tkinter + ttk 实现, 零外部依赖, 三个页签:

页签一「告警监控」:

- 按 `behavior_id` 分组树形展示告警列表, 行为头行灰色粗体标识
- 筛选栏: 检测器下拉 (全部/signature/bruteforce/anomaly)、严重度多选 (high/medium/low)、全文搜索输入框
- 点击任意告警行在右侧 JSON 详情面板显示完整告警记录
- 单条告警行按 `severity` 着色 (high 红/orange 橙/low 黄)
- 底部状态栏显示已加载告警总数 + 攻击行为事件数
- 30 秒定时自动刷新

页签二「特征库管理」:

- 表格展示 `config/signatures.txt` 中所有规则
- 工具栏: [+ 新增] [编辑] [删除] [导入] [导出] [保存到文件]
- 新增/编辑: 弹出 Toplevel 表单 (6 字段输入, combobox 下拉选择匹配模式和严重度)
- 导入: 文件选择对话框, 读取外部.txt 文件
- 导出/保存: 按 6 字段格式写回文件 (| 分隔)

页签三「统计概览」:

- Canvas 自绘柱状图 (攻击类型分布 + 严重度分布), 无需 matplotlib 等额外依赖
- 顶部摘要文字: 告警总数、攻击行为事件数、涉及攻击源 IP 数
- 颜色统一: high=#dc3545 红、medium=#fd7e14 橙、low=#ffc107 黄、图表柱=#4e73df 蓝

启动方式:

```
# 全链路运行 (检测+汇总+GUI)
python main.py --input mock_data/mock_packets.json
# 仅启动GUI (读取已有告警文件)
python main.py --gui-only
```

9 接口设计

所有模块通过两个标准化 JSON Schema 通信。详细规范见 `docs/interface_spec.md`。

9.1 报文记录格式 (Packet Record)

由 capture 模块产出, 三个检测模块统一消费。

```
{
  "timestamp": "2026-07-08T10:00:00.123",
  "flow_id": "192.168.1.10:51234->192.168.1.20:80/TCP",
  "src_ip": "192.168.1.10",
  "src_port": 51234,
  "dst_ip": "192.168.1.20",
  "dst_port": 80,
  "protocol": "TCP",
  "direction": "request",
  "flags": "PA",
  "payload": "GET /login.php?id=1 UNION SELECT ...",
  "payload_len": 128
}
```

表 6: Packet Record 字段定义

字段	类型	说明
timestamp	string(ISO8601)	报文捕获时间，精确到毫秒
flow_id	string	五元组流标识，格式 src_ip:src_port->dst_ip:dst_port/protocol
src_ip	string	源 IP 地址
src_port	int	源端口，ICMP/ARP 无端口协议为 null
dst_ip	string	目的 IP 地址
dst_port	int	目的端口
protocol	string	TCP / UDP / ICMP / ARP
direction	string	request / response
flags	string	TCP 标志位：S/A/P/F/R 组合
payload	string	应用层载荷，二进制用 \xNN 转义
payload_len	int	线路上原始 payload 字节长度

9.2 统一告警格式（Alert Record）

由三个检测模块产出，aggregator 消费。

```
{
  "alert_id": "b3f1a2c4-1234-4abc-9def-000000000001",
  "behavior_id": "b3f1a2c4-1234-4abc-9def-000000000001",
  "detector": "signature",
  "category": "Web 攻击/SQL注入",
  "src_ip": "192.168.1.10",
  "src_port": 51234,
  "dst_ip": "192.168.1.20",
  "dst_network": null,
  "dst_port": 80,
  "severity": "high",
  "description": "检测到针对 /login.php 的 SQL 注入攻击行为...",
  "evidence": "GET /login.php?id=1 UNION SELECT ...",
  "timestamp": "2026-07-08T10:00:01.000"
}
```


表 7: Alert Record 字段定义

字段	类型	说明
alert_id	string(uuid)	全局唯一告警 ID
behavior_id	string(uuid)	攻击行为事件标识（同源同类关联）
detector	string	signature / bruteforce / anomaly
category	string	攻击类型简述
src_ip	string	攻击源 IP
src_port	int	攻击源端口
dst_ip	string	受害目标 IP
dst_network	string	CIDR 网段（端口扫描/横向扩散时填写）
dst_port	int	受害目标端口
severity	string	low / medium / high
description	string	攻击行为描述 （行为导向语态，非特征罗列）
evidence	string	匹配原始内容或统计证据
timestamp	string(ISO8601)	告警产生时间

9.3 统一函数签名

所有检测模块必须遵循同一函数签名：

```
def detect(packets: list[dict]) -> list[dict]:
    """
    Args:
        packets: 符合 Packet Record 格式的列表
    Returns:
        符合 Alert Record 格式的列表（无告警时返回 [], 不返回 None）
    """
```

9.4 统一日志规范

所有模块使用 Python 标准库 logging：

```
import logging
logger = logging.getLogger(__name__)

logging.basicConfig(
    level=logging.INFO,
    format="[(name)s] %(levelname)s %(asctime)s %(message)s",
)
```

- logger.debug() — 调试信息（如跳过单条报文的原因）
- logger.info() — 正常流程信息（如“已加载 N 条规则”）
- logger.warning() — 非致命异常（文件缺失、字段缺失）

- `logger.error()` —致命错误（scapy 未安装）
- 禁止使用 `print()` 调试（CLI 结束时的摘要除外）

10 开发平台

10.1 软件平台

表 8: 软件平台选型

项目	选型	版本要求
开发语言	Python	≥ 3.9
抓包库	scapy	≥ 2.5.0
GUI 框架	tkinter	Python 标准库
测试框架	pytest	≥ 7.0
版本控制	Git + GitHub Flow	—
代码格式化	black + isort	≥ 24.0 / ≥ 5.13（推荐）

注意：不使用 Python 3.10+ 独有语法（如 `match/case`），保持向下兼容 Python 3.9。

10.2 硬件平台

- 最低配置：双核 CPU、4GB 内存、10GB 可用磁盘空间
- 推荐配置：四核 CPU、8GB 内存、SSD 硬盘
- 网络：实时抓包需网卡支持混杂模式
- 虚拟化：VirtualBox / VMware / WSL2 均可；虚拟机网络建议桥接模式
- 操作系统：Linux（推荐 Ubuntu 22.04 LTS）或 Windows 10/11 + WSL2

11 系统出错处理

表 9: 异常场景与处理策略

异常场景	处理策略	影响范围
输入文件不存在/为空	打印 WARNING 日志，返回空列表 []，不崩溃	单模块
报文记录缺失非必填字段	按 null 处理，跳过依赖该字段的判断逻辑	单条报文
payload 含无法解码的二进制	\xNN 转义保留，不影响其余字段	无
数据量不足无法形成统计窗口	不产生告警，不报错	无
flow_id 缺失	消费模块自行根据五元组构建	单模块内部
scapy 未安装	打印 ERROR 日志，返回空列表	capture 模块
时间戳无法解析	跳过该报文，其余报文正常处理	单条报文
JSON 文件格式异常	打印 WARNING/ERROR，跳过该文件	单文件
GUI 无告警数据	状态栏提示“未找到告警文件”，不弹错误对话框	GUI
特征库规则行字段数 $\neq 6$	跳过该行，打印 WARNING	单条规则
实时抓包权限不足	打印 ERROR，提示需 root/管理员权限	capture 模块
检测模块未产生告警	输出空 JSON 数组 [], aggregator 平滑处理	无

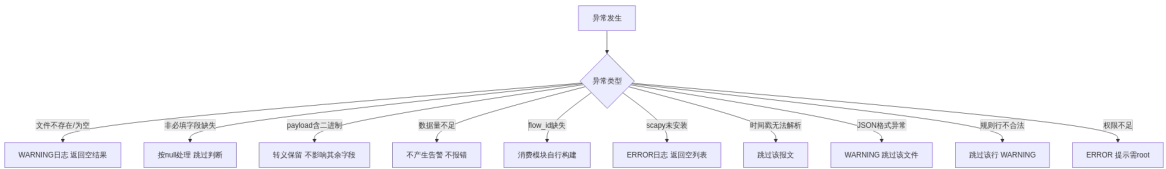


图 14: 系统出错处理决策树